

g·NEED ACCESS

NETWORK ENABLED EASY
DATA ACCESS



INSTRUCTIONS FOR USE



.NET CLIENT API



User Manual: g.NEEDaccess .NET Client API

Version 1.24.00 | Revision 1.0 | 2024

Copyright © g.tec medical engineering GmbH

www.gtec.at | office@gtec.at

CONTENTS

1 USING THE .NET CLIENT API.....	7
1.1 REFERENCE THE ASSEMBLY	7
1.2 EXECUTE TOGETHER WITH UNDERLYING C/C++ LIBRARIES	7
2 THE GDSCLIENTAPIDEMO PROJECT	8
2.1 COMPILING THE GDSCLIENTAPIDEMODOTNET PROJECT	8
2.2 READING THE RECORDED DATA FILE	8
3 CLIENT API – .NET REFERENCE	9
3.1 INTERFACE IGDSDEVICECONFIGURATION	9
3.2 CLASS GDSEXCEPTION : EXCEPTION	9
3.3 CLASS GDSCLIENTAPILIBRARYWRAPPER	10
3.3.1 CONSTANTS.....	10
DllName.....	10
DeviceNameLengthMax	10
3.3.2 ENUMERATIONS.....	10
ErrorCodes.....	10
GdsDeviceType.....	11
GdsFilterType.....	11
GdsChannelDirection	12
GdsElectrodeType	12
3.3.3 STRUCTURES.....	12
GdsResult	12
DeviceInfo.....	12
DeviceConnectionInfo	12
GdsConfiguration	13
GdsFilterInfo.....	13
3.3.4 DELEGATES	13
GdsCallback.....	13
GdsResultCallback.....	13
3.3.5 METHODS.....	15
IsDeviceLibraryLoadable	15
Initialize.....	16
Uninitialize	17
GetConnectedDevices.....	18
Connect	19
Disconnect.....	21
BecomeCreator	22
BecomeParticipant.....	23
SetConfiguration	24
GetConfiguration.....	25

StartAcquisition.....	26
StopAcquisition.....	27
StartStreaming.....	28
StopStreaming.....	29
GetDataInfo.....	30
GetData.....	31
SetConfigurationChangedCallback.....	33
SetDataAcquisitionStartedCallback.....	34
SetDataAcquisitionStoppedCallback.....	35
SetDataReadyCallback.....	36
SetDataAcquisitionErrorCallback.....	37
SetSessionCreatorDiedCallback.....	38
SetNewSessionCreatorCallback.....	39
SetServerDiedCallback.....	40
SetForcedClientShutdownCallback.....	41
3.4 CLASS GUSBAMPGDSCLIENTAPILIBRARYWRAPPER : GDSCLIENTAPILIBRARYWRAPPER....	42
3.4.1 CONSTANTS.....	42
MaxNumberOfChannels.....	42
MaxNumberOfGroups.....	42
MaxNumberOfAsyncDigitalIoChannels.....	42
3.4.2 ENUMERATIONS.....	42
GdsGUsbAmpWaveShape.....	42
3.4.3 STRUCTURES.....	42
GdsGUsbAmpAsyncDigitalIoChannel.....	42
GdsGUsbAmpScaling.....	43
GdsGUsbAmpSignalGeneratorConfiguration.....	43
GdsGUsbAmpChannelConfiguration.....	43
GdsGUsbAmpConfiguration : IGdsDeviceConfiguration.....	44
3.4.4 METHODS.....	47
GetDeviceInformation.....	47
GetBandpassFilters.....	48
GetNotchFilters.....	49
GetAsyncDigitalIOs.....	50
SetAsyncDigitalOutputs.....	51
GetScaling.....	52
SetScaling.....	53
Calibrate.....	54
GetImpedance.....	55
GetImpedanceEx.....	57
3.5 CLASS GHIAMPGDSCLIENTAPILIBRARYWRAPPER : GDSCLIENTAPILIBRARYWRAPPER	59
3.5.1 CONSTANTS.....	59
MaxNumberOfChannels.....	59
3.5.2 STRUCTURES.....	59

GdsGHiAmpScaling.....	59
GdsGHiAmpSignalGeneratorConfiguration.....	59
GdsGHiAmpChannelConfiguration.....	60
GdsGHiAmpConfiguration : IGdsDeviceConfiguration	60
3.5.3 METHODS.....	62
GetDeviceInformation.....	62
GetSupportedSamplingRates	63
GetBandpassFilters.....	64
GetNotchFilters	65
GetFactoryScaling.....	66
SetScaling.....	67
Calibrate	68
GetImpedance.....	69
GetImpedanceEx.....	71
3.6 CLASS GNAUTILUSGDSCLIENTAPILIBRARYWRAPPER : GDSCLIENTAPILIBRARYWRAPPER.74	
3.6.1 CONSTANTS.....	74
MaxNumberOfModules	74
NumberOfChannelsPerModule	74
MaxNumberOfChannels.....	74
AdditionalChannels.....	74
DigitalIoChannels	74
ElectrodeNameLengthMax.....	74
3.6.2 ENUMERATIONS.....	74
InputSignal	74
3.6.3 STRUCTURES.....	74
GdsGNautilusDigitalIoChannel	74
GdsGNautilusScaling	75
GdsGNautilusChannelConfiguration	75
GdsGNautilusConfiguration : IGdsDeviceConfiguration.....	76
3.6.4 DELEGATES	78
ImpedanceMeasurementCallback.....	78
3.6.5 METHODS.....	79
GetDeviceInformation.....	79
GetNumberOfMountedModules.....	80
GetChannelNames	81
GetAvailableChannels	82
GetAvailableDigitalIOs.....	83
GetSupportedSamplingRates	84
GetSupportedSensitivities	85
GetSupportedNetworkChannels.....	86
GetSupportedInputSources	87
SetNetworkChannel.....	88
GetNetworkChannel	89

GetScaling.....	90
SetScaling.....	91
Calibrate	92
GetBandpassFilters.....	93
GetNotchFilters	94
GetImpedance.....	95
GetImpedanceEx.....	96
StartImpedanceMeasurement.....	98
StartImpedanceMeasurementEx	100
StopImpedanceMeasurement.....	102

1 USING THE .NET CLIENT API

The .NET Client API library is a .NET assembly that wraps the C/C++ client API libraries, i.e. it provides a similar interface for .NET, but uses the underlying C/C++ libraries during execution. Thus, concepts, terminology and general usage are very similar.

See the g.NEEDaccess Client API documentation for details on general concepts, usage, and terminology.

1.1 REFERENCE THE ASSEMBLY

The only thing to do to integrate the g.NEEDaccess Client API in your application is to reference the g.NEEDaccess .NET Client API assembly. See the g.NEEDaccess Client API documentation for details on where to find the .NET assembly after installation.

See section Client API – .NET Reference for detailed descriptions of the provided namespaces, classes and methods. Use the table of contents for quick reference.

1.2 EXECUTE TOGETHER WITH UNDERLYING C/C++ LIBRARIES

When executing an application that uses g.NEEDaccess .NET Client API, the referenced .NET assembly must reside in the working directory of the application (which usually is the same directory that contains the application's executable itself).

Since the .NET assembly is just a wrapper of the C/C++ libraries and uses them underneath, the C/C++ dynamic link libraries (DLLs) must reside in that directory as well, or on a system search path during execution. They're required for execution only, not for compiling. If the same machine that runs the application also runs g.NEEDaccess Server, these libraries already reside on a system search path that the system's PATH environment variable points to, and don't need to be deployed additionally. See the g.NEEDaccess Client API documentation for details on the C/C++ libraries.

2 THE GDSCIENTAPIDEMO PROJECT

GDSCientAPIDemoDotNet is a console application illustrating the basic usage of g.NEEDaccess Client API in C++ programming language under the Microsoft Windows operating systems. Relevant source code and project files are located in the GDSCientAPIDemoDotNet folder, which is installed as an example for MS Windows. See the g.NEEDaccess Client API documentation on where to find examples after installation of g.NEEDaccess Client API.

A running instance of g.NEEDaccess Server on a host computer running Microsoft Windows OS is required to run the example. It is assumed that the example is executed at the same machine that runs the server. The example uses the loopback adapter address for communication with the server (see the g.NEEDaccess Server documentation for details). Furthermore, Microsoft Visual C# (part of Microsoft Visual Studio) is required to open the preconfigured project files and compile the examples' source code.

2.1 COMPILING THE GDSCIENTAPIDEMODOTNET PROJECT

Open the MS Visual C# project file provided in the GDSCientAPIDemoDotNet folder (`GdsClientAPIDemoDotNet.csproj`). Ensure that you have selected the appropriate platform setting (x86 for 32-bit, x64 for 64-bit) using the Configuration Manager from the Build menu, and compile it by selecting Build Solution from the Build menu. Currently, only 64-bit platforms (x64) are supported.

A folder named `bin` will be created in the project's folder containing the resulting executable in a subfolder according to the selected platform and configuration. Run the executable either from Visual Studio, by double-clicking it, or from the command line.

On startup, the application scans for supported devices connected to the current machine and prompts you to choose one of them from the presented list. The measurement procedure starts right after selecting a device and records data to a binary file (`receivedData.bin`). Data acquisition will stop as soon as the escape key (ESC) is pressed.

2.2 READING THE RECORDED DATA FILE

The recorded data file contains acquired samples as 32-bit float values. Data is organized as a concatenation of scans. Each scan consists of exactly one sample for each acquired channel, starting with the first acquired channel. All samples of a scan are sampled at the same point in time. It can be thought of a two-dimensional matrix in the format `channels x samples`.

The following piece of code illustrates how to read in the recorded data file in MATLAB. The `filename` variable must be set to the name of the recorded data file beforehand. The `channels` variable must be set to the number of acquired channels that the file contains beforehand, including the possibly recorded trigger channel (or other available additional channels). The number of acquired channels can be found via the method that sets up the configuration for the corresponding device type in the source code.

The data variable receives the recorded data from the file as a matrix in the format `channels x samples`.

```
filename = 'receivedData.bin';
channels = 16;
fid = fopen(filename, 'rb');
data = fread(fid, [channels, inf], 'float32');
fclose(fid);
```


3 CLIENT API – .NET REFERENCE

The .NET Client API is a library that provides all necessary methods for data acquisition using the server service. It consists of common methods that can be applied to any supported device, and additional methods for device-specific operations.

The .NET Client API can be used by any .NET application under Windows that represents a client. The namespace for each of the classes described below is:

Gtec.Gds.Client.API.Wrapper

3.1 INTERFACE IGdsDeviceConfiguration

Interface for concrete implementations of device-specific GDS configuration structures.

No members defined.

3.2 CLASS GdsException : Exception

The exception that is thrown when an error occurred using the GDS C-API.

Properties

- public [GdsClientApiLibraryWrapper.ErrorCodes](#) **ErrorCode**
Gets the error code of the device exception.

Constructors

- public **GdsException()**
Initializes a new instance of the [GdsException](#) class.
- public **GdsException(string message)**
Initializes a new instance of the [GdsException](#) class with a specified error message.
- public **GdsException(string message, Exception innerException)**
Initializes a new instance of the [GdsException](#) class with a specified error message and a reference to the inner exception that is the cause of this exception.
- public **GdsException([GdsClientApiLibraryWrapper.ErrorCodes](#) errorCode, string message)**
Initializes a new instance of the [GdsException](#) class with a specified error code and error message.
- public **GdsException([GdsClientApiLibraryWrapper.ErrorCodes](#) errorCode, string message, Exception innerException)**
Initializes a new instance of the [GdsException](#) class with a specified error code and error message and a reference to the inner exception that is the cause of this exception.
- public **GdsException([GdsClientApiLibraryWrapper.GdsResult](#) result)**
Initializes a new instance of the [GdsException](#) class using the specified [GdsClientApiLibraryWrapper.GdsResult](#) structure.
- public **GdsException([GdsClientApiLibraryWrapper.GdsResult](#) result, Exception innerException)**
Initializes a new instance of the [GdsException](#) class using the specified [GdsClientApiLibraryWrapper.GdsResult](#) structure and a reference to the inner exception that is the cause of this exception.

3.3 CLASS GdsClientApiLibraryWrapper

.NET-Wrapper providing access to the common native C-API of g.NEEDaccess (a.k.a. g.tec Device Service (GDS) - ClientAPI) from g.tec medical engineering GmbH.

See documentation of GDS - Client API for further information on the underlying C API functions.

Remarks

g.NEEDaccess (GDSClientAPI.dll and associates) for Microsoft Windows operating systems must be installed before.

3.3.1 CONSTANTS

DllName

Filename of the GDS - Client API dll-file.

DeviceNameLengthMax

The maximum number of characters a device name can contain, including the terminating null character.

3.3.2 ENUMERATIONS

ErrorCodes

The error codes that the GDS - ClientAPI can return.

Success	The operation completed successfully.
InvalidParameter	One of the specified parameters does not contain a valid value.
InvalidHandle	The specified connection handle is invalid.
HandleNotAssociated	The specified connection handle is not associated with any existing data acquisition session created or opened with Connect.
HandleAlreadyAssociated	The specified connection handle is already associated with an existing data acquisition session.
AccessDenied	An existing data acquisition session couldn't be opened because it has been opened exclusively by its creator, or the specified connection handle is not the creator of its associated data acquisition session, or the data acquisition session associated with the specified connection handle already has a creator.
InvalidDeviceList	The specified list of devices contains no or invalid entries.
InvalidDevice	A device cannot be part of the same data acquisition session than the others due to incompatible device type or settings, for example, or because the specified device is not part of the specified data acquisition session.
OpenDevice	A device could not be opened.
DeviceAlreadyInUse	A device is already part of an existing data acquisition session with a different list of associated devices, or an attempt was made to open an already existing data acquisition session exclusively.

InvalidConfiguration	A device could not be configured with the specified configuration, or is not configured at all.
IncomaptibleConfigurations	The device configurations are not compatible among themselves with respect to the associated data acquisition session.
ReadConfiguration	The configuration for a device could not be retrieved.
InvalidAcquisitionState	The operation is not allowed during the current acquisition state.
InvalidStreamingState	The operation is not allowed during the current streaming state.
BufferTooSmall	The specified number of elements to read exceeds the specified destination buffer's size.
AcquisitionBufferOverflow	An overflow occurred in the internal acquisition buffer of a data acquisition session. The server did not fetch data from the acquisition unit within a reasonable time. This indicates a problem on the server. The buffer will be reset automatically, and the client can continue data acquisition.
TransmissionBufferOverflow	An overflow occurred in a client's streaming buffer on the server. The client did not fetch data from the server in a reasonable time. The buffer will be reset automatically, and the client can continue data acquisition.
AcquisitionDied	An internal error occurred during a data acquisition session on the server forcing data acquisition for that session to stop. Connections are kept alive, and the creator of the concerned session can start data acquisition again with StartAcquisition
Serialization	A message or command could not be sent to the server because its content couldn't be serialized on the client side or deserialized. This indicates an internal error on the client or server side.
InvalidEndpoint	The specified endpoint does not exist, or no server is running there.
InvalidRequestStatus	The status of the request is invalid.
RequestTimedOut	The server did not reply to the sent request within a reasonable time.
UnknownError	An unknown error occurred

GdsDeviceType

The device types supported by the GDS.

NotSupported	Represents a not supported device type.
GUsbAmp	A g.USBamp device.
GHiAmp	A g.HIamp device.
GNautilus	A g.Nautilus device.

GdsFilterType

Types of filters.

Chebyshev	A Chebyshev filter.
Butterworth	A Butterworth filter.
Bessel	A Bessel filter.

GdsChannelDirection

Possible directions of a channel.

In	The channel is an input channel only.
Out	The channel is an output channel.

GdsElectrodeType

Types of electrodes.

Passive	A passive electrode.
Active	An active electrode.

3.3.3 STRUCTURES

GdsResult

Contains information about the success or failure of a GDS API method call.

Public Fields

- public [ErrorCodes](#) **ErrorCode**
The error code of the error that occurred, or [ErrorCodes](#). Success on success.

Properties

- public string **ErrorMessage**
Gets a message describing the error.

Member Functions

- public static implicit **operator** bool([GdsResult](#) result)
Returns a value indicating whether result's *ErrorCode* value equals [ErrorCodes](#).Success.

DeviceInfo

Holds information about a single connected device.

Public Fields

- public string **Name**
The name of the device.
- public [GdsDeviceType](#) **DeviceType**
The type of the device.

DeviceConnectionInfo

Holds information about the devices that belong to a single data acquisition session or no session at all.

Public Fields

- public bool **InUse**
Indicates whether the associated devices in *ConnectedDevices* are in use by a data acquisition unit instance.
- public List<[DeviceInfo](#)> **ConnectedDevices**
A list of connected devices that are in use by a single data acquisition unit instance or that are not in use at all, indicated by *InUse*.

GdsConfiguration

The configuration of a specific device.

Public Fields

- public [DeviceInfo](#) **DeviceInfo**
The device that this configuration belongs to.
- public [IGdsDeviceConfiguration](#) **DeviceConfiguration**
The configuration of a device.

GdsFilterInfo

Describes a filter.

Public Fields

- public [GdsFilterType](#) **TypeId**
The type of the filter.
- public uint **Order**
The order of the filter.
- public double **SamplingRate**
The sampling rate that the filter is designed for in Hertz.
- public double **LowerCutoffFrequency**
The lower cutoff frequency of the filter in Hertz.
- public double **UpperCutoffFrequency**
The upper cutoff frequency of the filter in Hertz.

3.3.4 DELEGATES

GdsCallback

References a method to be called when a parameterless GDS callback is triggered.

```
public delegate void GdsCallback(UInt64 connectionHandle, IntPtr userData)
```

Parameters

connectionHandle	The connection handle for which the callback was triggered.
userData	A pointer to an unmanaged block of memory containing the custom user data specified when setting the callback.

GdsResultCallback

References a method to be called when a parameterized GDS callback is triggered.

public delegate void GdsResultCallback(UInt64 connectionHandle, [GdsResult](#) result, IntPtr userData)

Parameters	
connectionHandle	The connection handle for which the callback was triggered.
result	Extended failure information.
userData	A pointer to an unmanaged block of memory containing the custom user data specified when setting the callback.

3.3.5 METHODS

IsDeviceLibraryLoadable

Determines whether all libraries required for device communication are installed.

```
public bool IsDeviceLibraryLoadable()
```

Returns

true if all required libraries are installed; otherwise, false.

Initialize

Initializes the GDS library.

```
public static void Initialize()
```

Remarks

[Initialize](#) must be called by a process before it uses any of the library functions. When a process is finished using the library, it must [Uninitialize](#) before the library is unloaded.

Uninitialize

Uninitializes the GDS library.

```
public static void Uninitialize()
```

Remarks

[Uninitialize](#) must be called before a process unloads the library. No library function must be called after calling [Uninitialize](#).

GetConnectedDevices

Gets a list of devices connected to the GDS server at the specified destination endpoint.

```
public static List<DeviceConnectionInfo> GetConnectedDevices(IPEndPoint destinationEndpoint,
IPEndPoint localEndpoint)
```

Parameters

destinationEndpoint	The remote endpoint of the GDS server that should return information about the devices connected to it.
localEndpoint	The local endpoint of the client that receives the reply from the server.

Returns

A list of [DeviceConnectionInfo](#) objects, each containing information about connected devices that belong to the same data acquisition session, or no data acquisition session at all.

Exceptions

GdsException	Couldn't retrieve information about connected devices from the specified endpoint.
------------------------------	--

Connect

Establishes a connection to the server, and creates a new session or opens an existing data acquisition session with the devices with the specified names.

```
public static UInt64 Connect(IPEndPoint destinationEndpoint, IPEndPoint localEndpoint, IList<string>
deviceNames, bool openExclusively, out bool isCreator)
```

Parameters

<code>destinationEndpoint</code>	The remote endpoint of the GDS server that the client should connect with.
<code>localEndpoint</code>	The local endpoint of the client that receives the reply from the server.
<code>deviceNames</code>	The list of device names identifying the devices which are connected to the server and part of the data acquisition to open or to create. Ordering matters. The first element in the array is considered the master device if multiple devices are hardware-synchronized. The others are considered slave devices in the specified order.
<code>openExclusively</code>	true to prevent a newly created data acquisition session from being opened or accessed by a second call to Connect with the same <code>deviceNames</code> list; otherwise, false.
<code>isCreator</code>	Receives a value indicating whether the caller is the creator or a participant of the data acquisition session.

Returns

The connection handle issued to the caller by the server.

Exceptions

<code>ArgumentException</code>	At least one element in <code>deviceNames</code> contains more than <code>DeviceNameLengthMax</code> characters.
GdsException	<code>deviceNames</code> is empty, or at least one device cannot be part of the same data acquisition session, or at least one device name in <code>deviceNames</code> refers to a device that couldn't be found or opened, or some device names in <code>deviceNames</code> are already part of another existing data acquisition session and the content of <code>deviceNames</code> is not equal to the list that was specified for that existing session on its creation, or an existing data acquisition should be opened but either <code>openExclusively</code> is set to true now or <code>openExclusively</code> was set to true on creation of the existing session to open.

Remarks

The returned connection handle is associated with the opened data acquisition session until [Disconnect](#) is called for that handle, or until the creator closes the data acquisition session.

Use [Disconnect](#) to release the connection. All open connections must be released before unloading the library.

A data acquisition session is identified by the list of device names that are part of the session. Each successful call of [Connect](#) receives a unique connection handle. The connection handle is used to identify a creator or participant of a data acquisition session and must be specified in almost every method of the API. Multiple connection handles can refer to the same data acquisition session that currently exists on a particular server.

When [Connect](#) is called for a specific list of device names the first time, a new data acquisition session is created for those devices, and data is acquired synchronously for all devices in that list. The returned connection handle is then referred to as the creator of the newly created data acquisition session.

If [Connect](#) is called with the exact same list of devices names that have already been opened by a previous call of [Connect](#), and that previous call did not open the devices exclusively (i.e. `openExclusively` was set to `false`), [Connect](#) succeeds and sets `isCreator` to `false`. The returned connection handle is then referred to as a participant of the existing data acquisition session.

[Connect](#) will fail if an existing data acquisition session should be opened, but the content of `deviceNames` is not exactly equal to the list of device names that was specified for that existing session when it was created (i.e. when [Connect](#) was called the first time for those device names).

Disconnect

Dissociates the specified connection handle from its associated data acquisition session and closes the connection to the server.

```
public static void Disconnect(ref UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle to dissociate and disconnect.
------------------	---

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

All registered callbacks for the specified connection handle will be cleared automatically.

If connectionHandle was the creator of its associated data acquisition session, the whole data acquisition session will be destroyed, all associated clients get disconnected and their **SessionCreatorDied** callback is executed if set.

Otherwise, only the specified connection handle gets dissociated, i.e. the data acquisition session remains running for all remaining connection handles that are still associated with it, and for which [Disconnect](#) has not been called before.



IMPORTANT

[Disconnect](#) must be called for each connection handle opened with [Connect](#) before the library is being unloaded. [Disconnect](#) must not be called from a process while that process is unloading the client API library because [Disconnect](#) is waiting for threads to complete, which would prevent the library from getting unloaded. This scenario may happen if [Disconnect](#) is called from a static (or singleton) class's destructor which is only called during library unloading.

BecomeCreator

Elevates the specified connection handle to the creator of its associated data acquisition session.

```
public static void BecomeCreator(UInt64 participantConnectionHandle)
```

Parameters

participantConnectionHandle	The connection handle of the participant that wants to get elevated to the creator of its associated data acquisition session.
-----------------------------	--

Exceptions

GdsException	participantConnectionHandle is invalid, or participantConnectionHandle is not associated with a data acquisition session opened with Connect before, or the data acquisition session associated with participantConnectionHandle already has an active creator.
------------------------------	---

Remarks

This method can be called for any participant of a data acquisition session if the creator of that session has died unexpectedly. Otherwise, this method will fail.

See [Connect](#) for details on the terms creator and participant of a data acquisition session.

The **NewSessionCreator** callback is raised for all participants of the associated data acquisition session on success.

BecomeParticipant

Downgrades the specified connection handle from the creator of its associated data acquisition session to a simple participant.

```
public static void BecomeParticipant(UInt64 creatorConnectionHandle)
```

Parameters

creatorConnectionHandle	The connection handle of the creator that should be downgraded to a participant of its associated data acquisition session.
-------------------------	---

Exceptions

GdsException	creatorConnectionHandle is invalid, or creatorConnectionHandle is not associated with a data acquisition session opened with Connect before, or creatorConnectionHandle is not creator of its associated data acquisition session.
------------------------------	--

Remarks

This method can only be called for creators of data acquisition sessions.

See [Connect](#) for details on the terms creator and participant of a data acquisition session.

The **SessionCreatorDied** callback is raised for all participants of the associated data acquisition session on success.

SetConfiguration

Sets the configuration for each device of the data acquisition session that was created with the specified connection handle.

```
public static void SetConfiguration(UInt64 connectionHandle, IList<GdsConfiguration>
deviceConfigurations)
```

Parameters

connectionHandle	The connection handle that the devices of its associated data acquisition session to configure has been created with.
deviceConfigurations	A list of configuration structures for each device that is part of the data acquisition session associated with the specified connection handle, in the order the corresponding device names have been specified by the previous call of Connect for connectionHandle.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or connectionHandle is not creator of its associated data acquisition session, or at least one entry in deviceConfigurations is invalid.
------------------------------	---

Remarks

The [GdsConfiguration.DeviceConfiguration](#) property of each element in deviceConfigurations must contain the appropriate derived type of [IGdsDeviceConfiguration](#) that exists for the corresponding device type. The device configuration structure at index i in deviceConfigurations must hold the configuration for the device in the device names list at index i that has been specified in the previous call of [Connect](#) for connectionHandle.

Each call of [SetConfiguration](#) must always contain all configurations for all devices of the associated data acquisition unit, and no other devices.

[SetConfiguration](#) can only be called for connection handles that are creators of their associated data acquisition sessions (see [Connect](#) for details on who is creator of a data acquisition session).

GetConfiguration

Gets the current configuration for each device of the associated data acquisition session.

```
public static IList<GdsConfiguration> GetConfiguration(UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle to return the configuration for the devices of its associated data acquisition session for.
------------------	---

Returns

A list of configuration structures for each device that is part of the data acquisition session associated with the specified connection handle.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

The [GdsConfiguration](#).DeviceConfiguration property of each element in the returned list is of the appropriate derived type of [IGdsDeviceConfiguration](#) that exists for the corresponding device type. [GetConfiguration](#) can be called for any connection handle that is associated with a specific data acquisition session (see [Connect](#) for details on who is creator of a data acquisition session).

StartAcquisition

Starts data acquisition on all devices of the data acquisition session that was created with the specified connection handle.

```
public static void StartAcquisition(UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle that the data acquisition session to start has been created with.
------------------	---

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or connectionHandle is not creator of its associated data acquisition session.
------------------------------	---

Remarks

Call [StartStreaming](#) afterwards to retrieve data.

Note that the behavior of [StartStreaming](#) is different from [StartAcquisition](#). While [StartAcquisition](#) actually starts data acquisition on the devices without streaming data to connected connection handles, [StartStreaming](#) enables streaming of acquired data to the connection associated with the specified connection handle. Hence, [StartStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [StartAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [Connect](#) for details on who is creator of a data acquisition session).

StopAcquisition

Stops data acquisition on all devices of the data acquisition session created with the specified connection handle.

```
public static void StopAcquisition(UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle used to create the data acquisition session to stop.
------------------	--

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or connectionHandle is not creator of its associated data acquisition session, or data acquisition is not running.
------------------------------	---

Remarks

Note that the behavior of [StopStreaming](#) is different from [StopAcquisition](#). While [StopAcquisition](#) actually stops data acquisition on the devices and streaming to all connected connection handles, [StopStreaming](#) disables streaming of acquired data just for the connection associated with the specified connection handle. Hence, [StopStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [StopAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [Connect](#) for details on who is creator of a data acquisition session).

StartStreaming

Enables data streaming for the specified connection handle.

```
public static void StartStreaming(UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle to start streaming for.
------------------	---

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or data acquisition is not running.
------------------------------	--

Remarks

[GetData](#) can be called afterwards to retrieve acquired data. connectionHandle must be associated with a data acquisition session with a preceding successful call to [Connect](#). Data acquisition must have been started by the creator with [StartAcquisition](#) for that session.

Note that the behavior of [StartStreaming](#) is different from [StartAcquisition](#). While [StartAcquisition](#) actually starts data acquisition on the devices without streaming data to connected connection handles, [StartStreaming](#) enables streaming of acquired data to the connection associated with the specified connection handle. Hence, [StartStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [StartAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [Connect](#) for details on who is creator of a data acquisition session).

StopStreaming

Disables data streaming for the specified connection handle.

```
public static void StopStreaming(UInt64 connectionHandle)
```

Parameters

connectionHandle	The connection handle to stop streaming for.
------------------	--

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	--

Remarks

Data acquisition remains running for connection handles that have streaming still enabled.

Note that the behavior of [StopStreaming](#) is different from [StopAcquisition](#). While [StopAcquisition](#) actually stops data acquisition on the devices and streaming to all connected connection handles, [StopStreaming](#) disables streaming of acquired data just for the connection associated with the specified connection handle. Hence, [StopStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [StopAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [Connect](#) for details on who is creator of a data acquisition session).

GetDataInfo

Returns information about the organization of the data to retrieve with [GetData](#) for the specified connection handle and the specified number of scans.

```
public static ulong GetDataInfo(UInt64 connectionHandle, ulong scanCount, out ulong[]
channelsPerDevice, out ulong bufferSizeInSamples)
```

Parameters

connectionHandle	The connection handle to retrieve information about the organization of the data that shall be retrieved.
scanCount	The number of scans to calculate the buffer size for, or zero to use the number of currently available scans instead.
channelsPerDevice	Returns an array with the number of channels that each device is acquiring, including synchronously sampled digital trigger channels. The number of elements in the array is equal to the total number of devices that are part of the data acquisition session associated with the specified connection handle. The element at index i corresponds to the device with serial number at index i in the device names list at index i that has been specified in the previous call of Connect for connectionHandle.
bufferSizeInSamples	Returns the number of samples for the returned number of scans. This is the number of elements a buffer must contain to be able to hold that amount of scans.

Returns

The number of scans for which the remaining output parameters were calculated. If scanCount is zero, the number of currently available scans is returned; otherwise, the returned value equals scanCount.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

A sample or element is the value of exactly one channel and is a 32-bit floating point number. A scan consists of exactly one sample for each acquired channel. The total number of acquired channels of all devices is the sum over all elements of channelsPerDevice, which is the number of elements of a single scan. The required size of the buffer in bytes to be able to hold the specified or returned number of scans is bufferSizeInSamples * sizeof(float).

GetData

Reads the specified number of scans into the specified destination buffer.

```
public static ulong GetData(UInt64 connectionHandle, ulong scanCount, IntPtr destinationBuffer,
    ulong destinationBufferSizeInSamples)
```

Parameters

connectionHandle	The connection handle to retrieve data for.
scanCount	The number of scans to read, or zero to read as many scans as either currently available or the specified destination buffer can contain, whichever is smaller.
destinationBuffer	A pointer to an allocated block of memory representing a float array, where read scans are written to. The size of the array must be large enough to receive the specified number of scans if scanCount is greater than zero.
destinationBufferSizeInSamples	The maximum number of elements of type float that the array in the memory block pointed to by destinationBuffer can contain.

Returns

The number of actually read scans.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or streaming has not been started for the specified connection handle, or scanCount is greater than zero and the specified size of the destination buffer is not large enough to receive the specified number of scans.
------------------------------	---

Remarks

Data is retrieved in microvolts as concatenated scans. A scan consists of exactly one sample for each acquired channel. A sample or element is the 32-bit floating point value of exactly one channel in microvolts. The value of sample *s* in channel *c* and a total number of *N* channels is stored in destinationBuffer[*s* * *N* + *c*]. The total number of acquired channels of all devices (i.e. the number of all elements in a single scan) can be determined by calling [GetDataInfo](#) with a scanCount value of 1.

The size of the destination buffer in bytes based on destinationBufferSizeInSamples would be destinationBufferSizeInSamples * sizeof(float).

If scanCount is greater than zero, this method verifies that the specified destination buffer is large enough to hold data for the specified number of scans using destinationBufferSizeInSamples. Otherwise, the number of scans to read is limited by the destination buffer's current size or the number of available scans, whichever is smaller.

If there are fewer scans available than specified by `scanCount`, the method reads only all available scans into the destination buffer, and returns the number of actually read scans.

Use [GetDataInfo](#) to determine the required destination buffer size for a specific number of scans. It is recommended to call [GetDataInfo](#) just once before streaming starts (or after the configuration has changed) for a specific amount of scans, allocate a fixed sized buffer and re-use this buffer for every consecutive [GetData](#) call without re-allocating it before each call. The buffer can be allocated with `Marshal.AllocHGlobal(int)`, or call `GCHandle.Alloc(object, GCHandleType)` with `GCHandleType.Pinned` on an allocated managed float array of size `destinationBufferSizeInSamples` and use `GCHandle.AddrOfPinnedObject()` to retrieve the pointer to the pinned block of memory that can be passed to `destinationBuffer`.



IMPORTANT

If a network or loopback adapter address is used to communicate with the server and [GetData](#) is called periodically with a fixed `scanCount` value that is too small (like 1...4 at 256 Hz sampling rate), the [GetData](#) request might be too slow to retrieve the data in realtime, and delays or data acquisition errors stating buffer overflows may occur.

SetConfigurationChangedCallback

Sets or clears the **ConfigurationChanged** callback that is triggered when the creator of a data acquisition session has changed the configuration of the associated devices.

```
public static void SetConfigurationChangedCallback(UInt64 connectionHandle, GdsCallback
configurationChangedCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the ConfigurationChanged callback for.
configurationChangedCallback	A delegate to the method to execute when the creator of a data acquisition session has changed the configuration of the associated devices, or null to clear the ConfigurationChanged callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetConfigurationChangedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **ConfigurationChanged** callback for the specified connection handle, set the configurationChangedCallback parameter to null.



IMPORTANT

configurationChangedCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to configurationChangedCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetDataAcquisitionStartedCallback

Sets or clears the **DataAcquisitionStarted** callback that is triggered when the creator of a data acquisition session has started acquisition.

```
public static void SetDataAcquisitionStartedCallback(UInt64 connectionHandle, GdsCallback
dataAcquisitionStartedCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the DataAcquisitionStarted callback for.
dataAcquisitionStartedCallback	A delegate to the method to execute when the creator of a data acquisition session has started acquisition, or null to clear the DataAcquisitionStarted callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetDataAcquisitionStartedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataAcquisitionStarted** callback for the specified connection handle, set the dataAcquisitionStartedCallback parameter to null.



IMPORTANT

dataAcquisitionStartedCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to dataAcquisitionStartedCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetDataAcquisitionStoppedCallback

Sets or clears the **DataAcquisitionStopped** callback that is triggered when the creator of a data acquisition session has stopped acquisition.

```
public static void SetDataAcquisitionStoppedCallback(UInt64 connectionHandle, GdsCallback
dataAcquisitionStoppedCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the DataAcquisitionStopped callback for.
dataAcquisitionStoppedCallback	A delegate to the method to execute when the creator of a data acquisition session has stopped acquisition, or null to clear the DataAcquisitionStopped callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetDataAcquisitionStoppedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataAcquisitionStopped** callback for the specified connection handle, set the dataAcquisitionStoppedCallback parameter to null.



IMPORTANT

dataAcquisitionStoppedCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to dataAcquisitionStoppedCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetDataReadyCallback

Sets or clears the **DataReady** callback that is triggered whenever the given amount of scans is available for the specified connection handle.

```
public static void SetDataReadyCallback(UInt64 connectionHandle, GdsCallback dataReadyCallback,
    UInt64 numberOfScans, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the DataReady callback for.
dataReadyCallback	A delegate to the method to execute when the given amount of scans is available, or null to clear the DataReady callback for the specified connection handle.
numberOfScans	The number of scans that must be retrieved during data acquisition at least in order to trigger the callback.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	--

Remarks

When [SetDataReadyCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataReady** callback for the specified connection handle, set the dataReadyCallback parameter to null.



IMPORTANT

dataReadyCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to dataReadyCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetDataAcquisitionErrorCallback

Sets or clears the **DataAcquisitionError** callback that is triggered when an error occurs during data acquisition.

```
public static void SetDataAcquisitionErrorCallback(UInt64 connectionHandle, GdsResultCallback dataAcquisitionErrorCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the DataAcquisitionError callback for.
dataAcquisitionErrorCallback	A delegate to the method to execute when an error occurs during data acquisition, or null to clear the DataAcquisitionError callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before. or numberOfScans equals zero and dataReadyCallback is not null.
------------------------------	--

Remarks

When [SetDataAcquisitionErrorCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one. To clear the **DataAcquisitionError** callback for the specified connection handle, set the dataAcquisitionErrorCallback parameter to null. The parameter of the callback method receives a [GdsResult](#) structure containing information about the error that has occurred during data acquisition.



IMPORTANT

dataAcquisitionErrorCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to dataAcquisitionErrorCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetSessionCreatorDiedCallback

Sets or clears the **SessionCreatorDied** callback that is triggered when the creator of a data acquisition session has died unexpectedly or when [BecomeParticipant](#) was called successfully.

```
public static void SetSessionCreatorDiedCallback(UInt64 connectionHandle, GdsCallback
sessionCreatorDiedCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the SessionCreatorDied callback for.
sessionCreatorDiedCallback	A delegate to the method to execute when the creator of a data acquisition session has died unexpectedly or when BecomeParticipant was called successfully, or null to clear the SessionCreatorDied callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetSessionCreatorDiedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **SessionCreatorDied** callback for the specified connection handle, set the sessionCreatorDiedCallback parameter to null.



IMPORTANT

sessionCreatorDiedCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to sessionCreatorDiedCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetNewSessionCreatorCallback

Sets or clears the **NewSessionCreator** callback that is triggered when a data acquisition session has a new creator (i.e. [BecomeCreator](#) was called successfully).

```
public static void SetNewSessionCreatorCallback(UInt64 connectionHandle, GdsCallback
newSessionCreatorCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the NewSessionCreator callback for.
newSessionCreatorCallback	A delegate to the method to execute when a data acquisition session has a new creator, or null to clear the NewSessionCreator callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetNewSessionCreatorCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **NewSessionCreator** callback for the specified connection handle, set the newSessionCreatorCallback parameter to null.



IMPORTANT

newSessionCreatorCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to newSessionCreatorCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetServerDiedCallback

Sets or clears the **ServerDied** callback that is triggered when the server was forced to shut down unexpectedly before all open connections were closed.

```
public static void SetServerDiedCallback(UInt64 connectionHandle, GdsCallback serverDiedCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the ServerDied callback for.
serverDiedCallback	A delegate to the method to execute when the server was forced to shut down unexpectedly before all open connections were closed, or null to clear the ServerDied callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	--

Remarks

When [SetServerDiedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **ServerDied** callback for the specified connection handle, set the serverDiedCallback parameter to null.



IMPORTANT

serverDiedCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to serverDiedCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

SetForcedClientShutdownCallback

Sets or clears the **ForcedClientShutdown** callback that is triggered when the server forced the connection associated with the specified connection handle to shut down unexpectedly.

```
public static void SetForcedClientShutdownCallback(UInt64 connectionHandle, GdsCallback forcedClientShutdownCallback, IntPtr userData)
```

Parameters

connectionHandle	The connection handle to set the ForcedClientShutdown callback for.
forcedClientShutdownCallback	A delegate to the method to execute when the server forced the connection associated with the specified connection handle to shut down unexpectedly, or null to clear the ForcedClientShutdown callback for the specified connection handle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before.
------------------------------	---

Remarks

When [SetForcedClientShutdownCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one. To clear the **ForcedClientShutdown** callback for the specified connection handle, set the forcedClientShutdownCallback parameter to null.



IMPORTANT

forcedClientShutdownCallback must be prevented from garbage collection for the lifetime of the connection (i.e. until [Disconnect](#) is called for connectionHandle or the callback is cleared) as long as the callback may be triggered. The recommended way to do this is to store a reference to the delegate passed to forcedClientShutdownCallback in a scope that exists until [Disconnect](#) is called or the callback is cleared.

3.4 CLASS GUsbAmpGdsClientApiLibraryWrapper : GdsClientApiLibraryWrapper

.NET-Wrapper providing access to the native C-API of g.NEEDaccess (a.k.a. g.tec Device Service (GDS) - ClientAPI) for g.USBamp devices from g.tec medical engineering GmbH.

See the documentation for GDS - Client API for further information on the underlying C API functions.

Remarks

g.NEEDaccess (GDSClntAPI.dll and associates) for Microsoft Windows operating systems must be installed before.

3.4.1 CONSTANTS

MaxNumberOfChannels

The maximum number of channels a g.USBamp device can provide.

MaxNumberOfGroups

The number of groups of a g.USBamp device that can be connected to separate or common ground and reference channels.

MaxNumberOfAsyncDigitalIoChannels

The number of asynchronous digital I/O channels a g.USBamp device provides.

Remarks

g.USBamp version 3.0 devices provide 4 digital output channels.

g.USBamp version 2.0 devices provide 2 digital input channels and 2 digital output channels.

3.4.2 ENUMERATIONS

GdsGUsbAmpWaveShape

The wave shapes provided by the internal signal generator.

Square	A square wave.
Sawtooth	A saw tooth.
Sine	A sine wave.
DRL	Driven Right Leg (DRL).
Noise	Noise.

3.4.3 STRUCTURES

GdsGUsbAmpAsyncDigitalIoChannel

Describes the state of an asynchronous digital I/O channel.

Public Fields

- public uint **ChannelNumber**
The one-based channel number of the digital I/O channel described by this instance.

- public [GdsChannelDirection](#) **Direction**
Indicates if the channel described by this instance is a digital input or output channel.
- public bool **Value**
The digital value of the channel.

Constructors

- public **GdsGUsbAmpAsyncDigitalloChannel** (uint channelNumber, [GdsChannelDirection](#) direction)
Initializes a new instance of the [GdsGUsbAmpAsyncDigitalloChannel](#) structure using the specified channel number and direction.
- public **GdsGUsbAmpAsyncDigitalloChannel**(uint channelNumber, [GdsChannelDirection](#) direction, bool value)
Initializes a new instance of the [GdsGUsbAmpAsyncDigitalloChannel](#) structure using the specified channel number, direction, and value.

GdsGUsbAmpScaling

The scaling values of each channel of a single g.USBamp device.

Public Fields

- public float[MaxNumberOfChannels] **Offset**
The offset for each channel in μV . The value at zero-based index i is the offset for one-based channel number $i + 1$.
- public float[MaxNumberOfChannels] **ScalingFactor**
The scaling factor for each channel. The value at zero-based index i is the scaling factor for one-based channel number $i + 1$.

GdsGUsbAmpSignalGeneratorConfiguration

The configuration of the internal signal generator.

Public Fields

- public bool **Enabled**
Indicates whether the internal signal generator should be enabled.
- public [GdsGUsbAmpWaveShape](#) **WaveShape**
The wave shape of the signal generator.
- public uint **Frequency**
The frequency of the signal generator in hertz. Valid values range from 1 Hz to 100 Hz.
- public int **Amplitude**
The amplitude of the signal generator in mV. Valid values range from -250 mV to +250 mV.
- public int **Offset**
The offset of the signal generator in mV. Valid values range from -200 mV to +200 mV.

Constructors

- public **GdsGUsbAmpSignalGeneratorConfiguration**(bool enabled, [GdsGUsbAmpWaveShape](#) waveShape, uint frequency, int amplitude, int offset)
Initializes a new instance of the [GdsGUsbAmpSignalGeneratorConfiguration](#) class using the specified configuration.

GdsGUsbAmpChannelConfiguration

The configuration of a single channel.

Public Fields

- public bool **Acquire**
Indicates whether the channel that this configuration belongs to should be included in data acquisition.
- public uint **BipolarChannel**
The bipolar derivation setting for the channel. Set this value to zero if unipolar derivation should be used with ground and reference electrodes that are connected to the ground and reference socket of the group that this channel belongs to. Otherwise, if bipolar derivation should be performed for the channel that this configuration belongs to, set *BipolarChannel* to the one-based channel number whose signal should be subtracted from the measured signal of this channel.
- public int **BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- public int **NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GetNotchFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.

Constructors

- public **GdsGUsbAmpChannelConfiguration**(bool acquire, int bandpassFilterIndex = -1, int notchFilterIndex = -1, uint bipolarChannel = 0)
Initializes a new instance of the [GdsGUsbAmpChannelConfiguration](#) class using the specified filters and bipolar channel.

GdsGUsbAmpConfiguration : IGdsDeviceConfiguration

The configuration of a g.USBamp device.

Public Fields

- public uint **SampleRate**
The sample rate of the analog inputs in hertz. Valid values are: 32, 64, 128, 256, 512, 600, 1200, 2400, 4800, 9600, 19200, 38400. Eight-channel g.USBamp devices support the following sample rates only: 128, 256, 512.
- public UIntPtr **NumberOfScans**
The number of scans to retrieve from the device at once, or zero to determine the recommended number of scans for the specified sampling rate automatically. Setting NumberOfScans to a value other than zero is recommended for advanced users only. This value depends on the value of SampleRate. The following table lists the recommended values of NumberOfScans for all possible sample rates.

Sampling Rate [Hz]	Recommended value for NumberOfScans
32	1
64	2
128	4
256	8

512	16
600	32
1200	64
2400	128
4800	256
9600	512
19200	512
38400	512

- public bool **ShortCutEnabled**
Indicates whether the short cut socket is enabled. If short cut is enabled, a HIGH level on the SC input socket of the amplifier disconnects the electrodes from the amplifier input stage as long as the HIGH level is applied. If short cut is disabled, the level on the SC input socket of the amplifier is ignored and the amplifier always delivers the currently measured values on its input channels.
- public bool **CounterEnabled**
Indicates whether a sample counter should be applied on channel 16 instead of the measured signal if selected for acquisition (overrun at 1,000,000 samples).
- public bool **TriggerEnabled**
Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition in an additional logical channel that is appended to the synchronously sampled analog input channels.
If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels.
g.USBamp version 2.0 has just one trigger line, so the values of the trigger channel can be 0 (LOW) or 250,000 (HIGH).
In version 3.0, there are 8 trigger lines coming bit-encoded as UINT8 value on the trigger channel. If all of the 8 trigger lines are HIGH, the value of the channel equals 255.0. If e.g. trigger line 0 to 3 are HIGH, the value of the channel equals 15.0.
- public [GdsGUsbAmpSignalGeneratorConfiguration](#) **InternalSignalGenerator**
The configuration of the internal signal generator.
- public bool[[MaxNumberOfGroups](#)] **CommonGround**
The common ground setting for each of the four groups A, B, C, and D.
Each element in the array indicates whether the ground socket of the corresponding group (A, B, C, or D) should be connected to common ground. The common ground setting for group A (channels 1 to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.
A value of false of an element in the array indicates that the ground socket of the corresponding group is not connected to common ground.
A value of true indicates that the ground socket of the corresponding group is connected to common ground.
- public bool[[MaxNumberOfGroups](#)] **CommonReference**
The common reference setting for each of the four groups A, B, C, and D. Each element in the array indicates whether the reference socket of the corresponding group (A, B, C, or D) should be connected to common reference. The common reference setting for group A (channels 1

to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.

A value of `false` of an element in the array indicates that the reference socket of the corresponding group is not connected to common reference.

A value of `true` indicates that the reference socket of the corresponding group is connected to common reference.

- public [GdsGUsbAmpChannelConfiguration](#)[[MaxNumberOfChannels](#)] **Channels**
The configuration of each channel. The element at zero-based index *i* holds the configuration of the channel with one-based channel number *i* + 1.

3.4.4 METHODS

GetDeviceInformation

Retrieves extended information about the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static string GetDeviceInformation(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to retrieve device information for.
deviceName	The name of the device to retrieve device information for.

Returns

The device information string.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetBandpassFilters

Returns the list of available bandpass filters for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsFilterInfo[] GetBandpassFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available bandpass filters for.
deviceName	The name of the device to get the list of available bandpass filters for.

Returns

A list of bandpass filters available for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetNotchFilters

Returns the list of available notch filters for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsFilterInfo[] GetNotchFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available notch filters for.
deviceName	The name of the device to get the list of available notch filters for.

Returns

A list of notch filters available for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetAsyncDigitalIOs

Returns the current values of all asynchronous digital input and output channels for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static IList<GdsGUsbAmpAsyncDigitalIoChannel> GetAsyncDigitalIOs(UInt64
connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to get the current asynchronous digital I/O channels' current values for.
deviceName	The name of the device to get the asynchronous digital I/O channels' current values for.

Returns

A list of the current values of all provided asynchronous digital input and output channels.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

SetAsyncDigitalOutputs

Sets or changes the values of the specified asynchronous digital output channels for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static void SetAsyncDigitalOutputs(UInt64 connectionHandle, string deviceName,
    IList<GdsGUsbAmpAsyncDigitalloChannel> asyncDigitalOutputs)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device whose asynchronous digital output channel values should be set or changed.
deviceName	The name of the device whose asynchronous digital output channel values should be set or changed.
asyncDigitalOutputs	The list of channels to modify together with their new values.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or asyncDigitalOutputs contains an entry for an invalid channel.

Remarks

Only the output channels that are specified in the asyncDigitalOutputs list are modified; others preserve their current state.

g.USBamp version 3.0, devices accept channel numbers from 1 to 4.

g.USBamp version 2.0, devices accept channel numbers from 1 to 2.

GetScaling

Returns the scaling values for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsGUsbAmpScaling GetScaling(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to get the scaling values for.
asyncDigitalOutputs	The list of channels to modify together with their new values.
deviceName	The name of the device to get the scaling values for.

Returns

The scaling values for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

SetScaling

Sets the scaling values for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static void SetScaling(UInt64 connectionHandle, string deviceName, GdsGUsbAmpScaling scaling)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	The name of the device to set the scaling values for.
scaling	The scaling values to set.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Calibrate

Calculates and returns new scaling values for each channel under the current conditions of the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

public static [GdsGUsbAmpScaling](#) Calibrate(UInt64 connectionHandle, string deviceName)

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to calibrate.
deviceName	The name of the device to calibrate.

Returns

The calculated scaling values for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

The calculated and returned scaling values are not applied on the device yet. Use [SetScaling](#) with the returned scaling structure to apply them on the device.

GetImpedance

Measures and returns the impedance at the specified channel of the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

This method is deprecated (use [GetImpedanceEx](#) instead).

```
public static double GetImpedance(UInt64 connectionHandle, string deviceName, byte channel)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to use for impedance measurement.
deviceName	The name of the device to use for impedance measurement.
channel	The one-based number of the channel whose impedance should be measured (see remarks section for details).

Returns

The impedance value of the specified channel in ohm.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running, or channel specifies an invalid channel number.

Remarks

All grounds are connected to common ground during impedance measurement. Impedance is measured between the specified channel and ground.

Impedance can be measured from ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for channel1:

Value for channel	Description
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

GetImpedanceEx

Measures and returns the impedance at the specified channel of the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

```
public static double GetImpedanceEx(UInt64 connectionHandle, string deviceName, byte channel)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to use for impedance measurement.
deviceName	The name of the device to use for impedance measurement.
channel	The one-based number of the channel whose impedance should be measured (see remarks section for details).

Returns

The impedance value of the specified channel in ohm.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running, or channel specifies an invalid channel number.

Remarks

All grounds are connected to common ground during impedance measurement. Impedance is measured between the specified channel and ground.

Impedance can be measured from ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for channel1:

Value for channel	Description
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

All returned impedance values are rounded to the nearest integral ohm value. Negative values represent different error codes.

The following table lists the recommended classification of returned impedances and error codes for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

Impedances for active electrodes connected via a g.GAMMAbox cannot be measured with g.USBamp devices.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The impedance could not be calculated but is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The impedance could not be calculated but is estimated to be very high.

3.5 CLASS GHiAmpGdsClientApiLibraryWrapper :

GdsClientApiLibraryWrapper

.NET-Wrapper providing access to the native C-API of g.NEEDaccess (a.k.a. g.tec Device Service (GDS) - ClientAPI) for g.HIamp devices from g.tec medical engineering GmbH.

See documentation of GDS - Client API for further information on the underlying C API functions.

Remarks

g.NEEDaccess (GDSCClientAPI.dll and associates) for Microsoft Windows operating systems must be installed before.

3.5.1 CONSTANTS

MaxNumberOfChannels

The maximum number of channels a g.HIamp device provides.

3.5.2 STRUCTURES

GdsGHiAmpScaling

The scaling values of each channel of a single g.USBamp device.

Public Fields

- public float[[MaxNumberOfChannels](#)] **Offset**
The offset for each channel in μV . The value at zero-based index i is the offset for one-based channel number $i + 1$.
- public float[[MaxNumberOfChannels](#)] **ScalingFactor**
The scaling factor for each channel. The value at zero-based index i is the scaling factor for one-based channel number $i + 1$.

GdsGHiAmpSignalGeneratorConfiguration

The configuration of the internal signal generator.

Public Fields

- public bool **Enabled**
Indicates whether the internal signal generator should be enabled.
- public ushort **Frequency**
The frequency of the signal generator in hertz. Valid values range from 1 Hz to 700 Hz.

Properties

- public double **Amplitude**
Gets the amplitude of the signal generator in μV .
- public double **Offset**
Gets the offset of the signal generator in μV .

Constructors

- public **GdsGHiAmpSignalGeneratorConfiguration**(bool enabled, ushort frequency)
Initializes a new instance of the [GdsGHiAmpSignalGeneratorConfiguration](#) class using the specified configuration.

Remarks

The internal signal generator emits a square-wave signal with an amplitude of +/- 7622.83 μ V and an offset of -7622.83 μ V.

GdsGHiAmpChannelConfiguration

The configuration of a single channel.

Public Fields

- public bool **Acquire**
Indicates whether the channel this configuration belongs to should be included in data acquisition.
- public int **BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- public int **NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GetNotchFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- public ushort **ReferenceChannel**
The one-based number of the channel to use as reference channel for the current channel, or zero if no reference should be used for the current channel.

Constructors

- public **GdsGHiAmpChannelConfiguration**(bool acquire, ushort referenceChannel = 0, int bandpassFilterIndex = -1, int notchFilterIndex = -1)
Initializes a new instance of the [GdsGHiAmpChannelConfiguration](#) class using the specified reference channel and filters.

GdsGHiAmpConfiguration : IGdsDeviceConfiguration

The configuration of a g.HIamp device.

Public Fields

- public uint **SamplingRate**
The sampling rate of the synchronously sampled inputs in hertz. Use [GetSupportedSamplingRates](#) for a list of valid values.
- public ushort **NumberOfScans**
The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting *NumberOfScans* to a value other than zero is recommended for advanced users only. This value depends on the value of *SamplingRate*. The following table lists the recommended values of *NumberOfScans* for several sampling rates.

Sampling Rate [Hz]	Recommended value for NumberOfScans
256	8
512	16
600	32
1200	64
2400	128
4800	256
9600	256
19200	256
38400	256

- public bool **TriggerLinesEnabled**
Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition. If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels. For g.HIamp, there are 16 trigger lines encoded in this value. The eight least significant bits (bits 1 to 8) contain the digital values (0 for LOW or 1 for HIGH) of trigger channels 1 to 8. Bits 17 to 24 contain the digital values of trigger channels 9 to 16. Bits 9 to 16 and bits 25 to 32 of the value are not used.
- public bool **HoldEnabled**
Indicates whether the signal hold input is enabled. If the signal hold input is enabled, a HIGH level on the HOLD input causes the amplifier to latch the input channels' current values and ignore the actually applied signal until the HOLD input is pulled down to LOW again. If the signal hold input is disabled, the signal applied to the HOLD input has no effect.
- public bool **CounterEnabled**
Indicates whether a sample counter should be applied on the very first channel selected for acquisition. If enabled, a sample counter is applied to the very first channel that is selected for acquisition. The actually applied signal on that channel is disregarded then. The counter overruns at 1,000,000 samples.
- public [GdsGHIampSignalGeneratorConfiguration](#) **InternalSignalGenerator**
The configuration of the internal signal generator.
- public [GdsGHIampChannelConfiguration](#)[[MaxNumberOfChannels](#)] **Channels**
The configuration of each channel. An element at zero-based index i holds the configuration of the channel with one-based channel number i + 1. Channels that the device does not provide cannot be selected for acquisition. The other settings in their configuration structure will be ignored.

3.5.3 METHODS

GetDeviceInformation

Retrieves extended information about the specified g.Hlamp device of the data acquisition session associated with the specified connection handle.

```
public static string GetDeviceInformation(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to retrieve device information for.
deviceName	The name of the device to retrieve device information for.

Returns

The device information string.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetSupportedSamplingRates

Retrieves the supported sampling rates of a g.Hlamp device.

```
public static List<uint> GetSupportedSamplingRates(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of supported sampling rates for.
deviceName	The name of the device to get the list of supported sampling rates for.

Returns

A list of available sampling rates for the device in hertz.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetBandpassFilters

Returns the list of available bandpass filters for the specified g.Hlamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsFilterInfo[] GetBandpassFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available bandpass filters for.
deviceName	The name of the device to get the list of available bandpass filters for.

Returns

A list of bandpass filters available for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetNotchFilters

Returns the list of available notch filters for the specified g.Hlamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsFilterInfo[] GetNotchFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available notch filters for.
deviceName	The name of the device to get the list of available notch filters for.

Returns

A list of notch filters available for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetFactoryScaling

Returns the scaling values initially set by the manufacturer for the specified g.HIamp device of the data acquisition session associated with the specified connection handle.

```
public static GdsGHiAmpScaling GetFactoryScaling(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to get the factory scaling values for.
deviceName	The name of the device to get the factory scaling values for.

Returns

The factory scaling values for the specified device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

SetScaling

Sets the scaling values for the specified g.HIamp device of the data acquisition session associated with the specified connection handle.

```
public static void SetScaling(UInt64 connectionHandle, string deviceName, GdsGHIampScaling scaling)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	The name of the device to set the scaling values for.
scaling	The scaling values to set.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

[Calibrate](#) may return NaN values for channels it cannot calculate scaling values for.

Any Single.NaN values in scaling's [GdsGHIampScaling](#).ScalingFactor array will be replaced by 1 before they're sent to the device.

Any Single.NaN values in scaling's [GdsGHIampScaling](#).Offset array will be replaced by 0 before they're sent to the device.

Calibrate

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions if the calibration set is applied.

public static [GdsGHiAmpScaling](#) Calibrate(UInt64 connectionHandle, string deviceName)

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to calibrate.
deviceName	The name of the device to calibrate.

Returns

The new scaling values calculated under the current conditions for each synchronously sampled analog input channel.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

All channels must be shorted to ground (GND). This is done by applying the calibration equipment, consisting of four connectors, one for each group (A, B, C, and D). They connect all channel inputs of a group with ground (GND).

The returned values will not be applied unless they're explicitly set with [SetScaling](#).

Calibration is required if amplitude and/or offset of the known input signal differs from the measured signal. The function returns `Single.NaN` as scaling factor and offset for channels that failed calibration.

GetImpedance

Returns the impedance of all available synchronously sampled analog input channels for active or passive electrodes in ohm.

This method is deprecated (use [GetImpedanceEx](#) instead).

```
public static IDictionary<ushort, double> GetImpedance(UInt64 connectionHandle, string
deviceName, IList<ushort> activeElectrodeChannels)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.
activeElectrodeChannels	A list of one-based channel numbers that active electrodes are connected to. All remaining channels are assumed to have passive electrodes connected.

Returns

A dictionary containing one-based channel numbers in its keys and the corresponding impedance value in ohm in its values.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
ArgumentOutOfRangeException	At least one element in activeElectrodeChannels is less than 1 or greater than MaxNumberOfChannels .
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running, or activeElectrodeChannels is not empty but does not contain all available channels (active and passive electrode types are not allowed to be mixed).

Remarks

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used. Active and passive electrodes cannot be mixed. The method will throw an exception if mixed electrode types are specified.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will never be included in the returned dictionary. The method will not throw an exception if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

All returned impedance values are rounded to the nearest integral kilo-ohm value. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. Values below 5,000 ohm or above 500,000 ohm represent different error codes and do not rate the severeness of error. Thus, they are not comparable to each other. The returned map only includes available channels that yielded a finite impedance value, except channel 1.

If a TMS box is connected, the caller must subtract 10 kilo-ohm from valid impedance values to get the correct value (do not do this for error codes).

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 5,000	Not measurable, estimated good impedance.
5,000...500,000	Measured impedance values.
> 500,000	Not measurable impedance due to an erroneous signal (maybe due to a not connected electrode, a too big offset, etc.).

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
3,000	The impedance could not be calculated but is estimated to be very low.
4,000	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
501,000	The amplitude of the measured signal is too small.
502,000	The offset of the measured signal is too large.
503,000	The RMS value of the measured signal is too large.
504,000	The low frequency power of the measured signal is too large.
505,000	The amplitude of the measured signal at channel 1 is too small.
506,000	The offset, RMS, or low frequency power at channel 1 is too large.

GetImpedanceEx

Returns the impedance of all available synchronously sampled analog input channels for active or passive electrodes in ohm.

```
public static IDictionary<ushort, double> GetImpedanceEx(UInt64 connectionHandle, string
deviceName, GdsElectrodeType electrodeType, bool tmsBoxCorrection, bool
firstMeasurementOfSession)
```

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.
electrodeType	The type of connected electrodes. All electrodes must be of the same selected type.
tmsBoxCorrection	true if a TMS electrode connector box is connected and impedance values of all channels should be corrected by 10 kilo-ohm; otherwise, false.
firstMeasurementOfSession	If this is the start of a new measurement session (i.e. the first in a series of consecutive calls to GetImpedanceEx), set this flag to true to clear internal interim values of previous measurements; for the following repeated calls, set this flag to false until the end of the measurement session.

Returns

A dictionary containing one-based channel numbers in its keys and the corresponding impedance value in ohm in its values.

Exceptions	
ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
ArgumentOutOfRangeException	At least one element in activeElectrodeChannels is less than 1 or greater than MaxNumberOfChannels .
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used. Active and passive electrodes cannot be mixed.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will never be included in the returned dictionary. The method will not throw an exception if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

If a TMS electrode connector box is connected, set the `tmsBoxCorrection` parameter to `true`; 10 kilo-ohm will be subtracted of all measured impedance values in this case due to internal dropping resistors. Hence, no other type of electrode connector boxes can be connected to the same device if an TMS electrode connector box is connected for impedance measurement.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The returned map only includes available channels that yielded a finite impedance value, except channel 1.

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.

-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too big.
-5	The RMS value of the measured signal is too big.
-6	The drift of the measured signal is too big.
-9	The amplitude of the measured signal at channel 1 is too small.
-10	The offset, RMS, or drift at channel 1 is too big.

3.6 CLASS GNautilusGdsClientApiLibraryWrapper : GdsClientApiLibraryWrapper

.NET-Wrapper providing access to the native C-API of g.NEEDaccess (a.k.a. g.tec Device Service (GDS) - ClientAPI) for g.Nautilus devices from g.tec medical engineering GmbH.

See documentation of GDS - Client API for further information on the underlying C API functions.

Remarks

g.NEEDaccess (GDSClientAPI.dll and associates) for Microsoft Windows operating systems must be installed already.

3.6.1 CONSTANTS

MaxNumberOfModules

The maximum number of modules a g.Nautilus device provides.

NumberOfChannelsPerModule

The number of channels a single module hosts.

MaxNumberOfChannels

The maximum number of electrode channels a g.Nautilus device provides.

AdditionalChannels

The maximum number of channels a g.Nautilus device provides in addition to the electrode channels (e.g.: counter, acceleration data, ...).

DigitalloChannels

The number of asynchronous digital input channels a g.Nautilus device provides.

ElectrodeNameLengthMax

The maximum length of an electrode name.

3.6.2 ENUMERATIONS

InputSignal

An enumeration of possible input signals for a channel of a g.Nautilus device.

Electrode	Analog input from electrodes.
Shortcut	Channel inputs shorted to GND.
TestSignal	A test signal is the source.

3.6.3 STRUCTURES

GdsGNautilusDigitalloChannel

Describes the state of an asynchronous digital I/O channel.

Public Fields

- public uint **ChannelNumber**
The one-based channel number of the digital IN channel described by this instance.
- public [GdsChannelDirection](#) **Direction**
Indicates whether the channel described by this instance is an input or output channel.

Constructors

- public **GdsGNautilusDigitalloChannel**(uint channelNumber, [GdsChannelDirection](#) direction)
Initializes a new instance of the [GdsGNautilusDigitalloChannel](#) structure using the specified channel number and direction.

GdsGNautilusScaling

The scaling values for each channel of a single g.Nautilus device.

Public Fields

- public float[[MaxNumberOfChannels](#)] **Offset**
The offset for each channel in μV . The value at zero-based index i is the offset for zero-based channel number i .
- public float[[MaxNumberOfChannels](#)] **Factor**
The scaling factor for each channel. The value at zero-based index i is the scaling factor for zero-based channel number i .

GdsGNautilusChannelConfiguration

The configuration of a single channel.

Public Fields

- public bool **Enabled**
Indicates whether the channel is enabled for data acquisition.
- public double **Sensitivity**
The input sensitivity of the channel in microvolts.
- public bool **UsedForNoiseReduction**
Indicates whether the channel is used for noise reduction computation or not.
- public bool **UsedForCAR**
Indicates whether the channel is used for common average reference (CAR) calculation.
- public int **BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- public int **NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GetNotchFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- public int **BipolarChannel**
The bipolar derivation setting for the channel. Set this value to -1 if unipolar derivation should be used with ground and reference electrodes. Otherwise, if bipolar derivation should be performed for the channel that this configuration belongs to, set *BipolarChannel* to the zero-based channel number whose signal should be subtracted from the measured signal of this channel.

Constructors

- **public GdsGNautilusChannelConfiguration**(bool enabled, double sensitivity = 187500.0, bool usedForNoiseReduction = false, bool usedForCAR = false, int bandpass = -1, int notch = -1, int bipolar = -1)
Initializes a new instance of the [GdsGNautilusChannelConfiguration](#) class using the specified parameters.

GdsGNautilusConfiguration : IGdsDeviceConfiguration

The configuration of a g.Nautilus device.

Public Fields

- **public bool IsSlave**
Indicates whether the g.Nautilus device shall be operated in slave mode or not.
- **public [InputSignal](#) InputSignal**
The input signal to use.
- **public uint SamplingRate**
The sampling rate of the synchronously sampled inputs in hertz. Use [GetSupportedSamplingRates](#) for a list of valid values.
- **public uint NumberOfScans**
The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting *NumberOfScans* to a value other than zero is recommended for advanced users only. This value depends on the value of *SamplingRate*. The following table lists the recommended values of *NumberOfScans* for several sampling rates.

Sampling Rate [Hz]	Recommended value for NumberOfScans
250	8
500	15

- **public bool NoiseReduction**
Indicates whether noise reduction for the g.Nautilus device is enabled or not.
- **public bool CAR**
Indicates whether common average (CAR) computation for the g.Nautilus device is enabled or not.
- **public bool AccelerationData**
Indicates whether the acquisition of acceleration data is enabled or not for the g.Nautilus device. If enabled, acceleration values of the x, y, and z axis come as three additional channels appended to the analog channels. Acceleration values range within ± 6 g.
- **public bool Counter**
Indicates whether the acquisition of the counter is enabled or not for the g.Nautilus device. If enabled, counter comes as an additional channel appended to the acceleration data channels. The counter value is the number of the current sample.
- **public bool LinkQualityInformation**
Indicates whether the acquisition of the link quality information is enabled or not for the g.Nautilus device. If enabled, link quality information comes as an additional channel appended to the counter channel. Link quality information values range within 0 and 100 percent.

- public bool **BatteryLevel**
Indicates whether the acquisition of the battery level (state of charge, SOC) is enabled or not for the g.Nautilus device. If enabled, battery level comes as an additional channel appended to the link quality information channel. Battery level values range within 0 and 100 percent.
- public bool **DigitalIOs**
Indicates whether the acquisition of the digital IOs is enabled or not for the g.Nautilus device. If enabled, the values of the digital trigger input channels come bit-encoded in a single additional channel appended to the battery level channel.
- public bool **ValidationIndicator**
Indicates whether the acquisition of the validation indicator is enabled or not for the g.Nautilus device. If enabled, the value of the validation indicator comes as an additional channel appended to the digital IO channel. Valid validation indicator values are 0 and 1.
- public uint **NetworkChannel**
The number of the network channel the head set and base station of a g.Nautilus device shall use to communicate. Zero indicates the device chooses the channel automatically.
- public [GdsGNautilusChannelConfiguration](#)[[MaxNumberOfChannels](#)] **Channels**
An array where each element contains the configuration for the corresponding channel. An element at zero-based index i holds the configuration of the channel with zero-based channel number i. Channels that the device does not provide cannot be selected for acquisition. The other settings in their configuration structures will be ignored.

Constructors

- public **GdsGNautilusConfiguration**([GdsGNautilusChannelConfiguration](#)[] channels, bool isSlave = false, [InputSignal](#) inputSignal = [GNautilusGdsClientApiLibraryWrapper.InputSignal.Electrode](#), uint samplingRate = 250, uint numberOfScans = 0, bool noiseReduction = false, bool car = false, bool accelerationData = true, bool counter = true, bool linkQualityInformation = true, bool batteryLevel = true, bool digitalIOs = true, bool validationIndicator = true, uint networkChannel = 0)
Initializes a new instance of [GdsGNautilusConfiguration](#) using the specified parameters.

3.6.4 DELEGATES

ImpedanceMeasurementCallback

References a method to be called whenever an asynchronous impedance measurement cycle has completed.

```
public delegate void ImpedanceMeasurementCallback(UInt64 connectionHandle, string deviceName,
[MarshalAs(UnmanagedType.LPArray, SizeConst = MaxNumberOfChannels)] double[] impedances,
[MarshalAs(UnmanagedType.Bool)] bool aborted, GdsResult result, IntPtr userData)
```

Parameters	
connectionHandle	The connection handle for which the callback was triggered.
deviceName	The name of the device that triggered the callback.
impedances	An allocated array that receives the impedance values in ohm for each channel. An element at zero-based index i receives the impedance value in ohm of zero-based channel number i, or Double.NaN (see extended description of GetImpedanceEx/GetImpedance for details).
aborted	Indicates whether any error occurred that is crucial for impedance measurement so that measurement had to be stopped automatically.
result	Extended failure information, if an error occurred.
userData	A pointer to an unmanaged block of memory containing the custom user data specified when setting the callback.

3.6.5 METHODS

GetDeviceInformation

Retrieves extended information about the specified device.

```
public static string GetDeviceInformation(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve extended device information for.
deviceName	The name of the device to retrieve extended device information for.

Returns

The extended device information string.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetNumberOfMountedModules

Gets the number of the number of actually mounted modules of the specified g.Nautilus device.

```
public static uint GetNumberOfMountedModules(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the number of mounted modules for.
deviceName	The name of the device to retrieve the number of mounted modules for.

Returns

The number of actually mounted modules.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

This is device specific and invariant. The maximum number of modules a g.Nautilus device can host is [MaxNumberOfModules](#).

GetChannelNames

Gets the names of all available channels depending on the mounted grid.

```
public static Dictionary<uint, string> GetChannelNames(UInt64 connectionHandle, string
deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the naming of the channels for.
deviceName	The name of the device to retrieve the naming of the channels for.

Returns

A dictionary containing a channel name for each available zero-based channel number.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetAvailableChannels

Returns a boolean flag, for each possible channel a device in its full configuration may provide, indicating whether the corresponding channel is actually provided by and available on the device.

```
public static bool[] GetAvailableChannels(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available channels for.
deviceName	The name of the device to get the list of available channels for.

Returns

An array of length [MaxNumberOfChannels](#) where an element at index i receives a boolean flag indicating whether the corresponding channel with zero-based channel number i is available on the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetAvailableDigitalIOs

Gets information about available digital I/O channels of the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

```
public static IList<GdsGNautilusDigitalIoChannel> GetAvailableDigitalIOs(UInt64 connectionHandle,
string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to get information about available digital I/O channels for.
deviceName	The name of the device to get information about available digital I/O channels for.

Returns

A list of all provided asynchronous digital input and output channels.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetSupportedSamplingRates

Retrieves the supported sampling rates of a g.Nautilus device.

```
public static List<UInt32> GetSupportedSamplingRates(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the supported sampling rates for.
deviceName	The name of the device to retrieve the supported sampling rates for.

Returns

A list of available sampling rates for the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetSupportedSensitivities

Retrieves the supported sensitivity values for a g.Nautilus device.

```
public static List<double> GetSupportedSensitivities(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the supported sensitivities for.
deviceName	The name of the device to retrieve the supported sensitivities for.

Returns

A list of the supported sensitivities of a g.Nautilus device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetSupportedNetworkChannels

Retrieves the available network channels a g.Nautilus device can use to pair its headset and base station.

```
public static List<UInt32> GetSupportedNetworkChannels(UInt64 connectionHandle, string
deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the supported network channels for.
deviceName	The name of the device to retrieve the supported network channels for.

Returns

A list of available network channels for pairing.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

Usually a g.Nautilus device pairs itself automatically, but if needed to set the network channel manually, it has to be one of these.

GetSupportedInputSources

Retrieves the available input sources a g.Nautilus device can use to acquire signals.

```
public static List<InputSignal> GetSupportedInputSources(UInt64 connectionHandle, string
deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the supported input sources for.
deviceName	The name of the device to retrieve the supported input sources for.

Returns

A list of available input sources for the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

SetNetworkChannel

Sets the radio channel to use for wireless communication between the headset and base station.

```
public static void SetNetworkChannel(UInt64 connectionHandle, string deviceName, uint
networkChannel)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to set the network channel for.
deviceName	The name of the device to set the current network channel for.
networkChannel	The specified network channel number that shall be set for the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

The network channel number specifies the channel that is used for communication between its headset and base station. When switching on a g.Nautilus device, it pairs itself automatically to a base station using the first free channel selected by the base station. A free channel is a channel where no communication is running.

Use [GetSupportedNetworkChannels](#) to retrieve a list of supported network channels.

GetNetworkChannel

Retrieves the current radio channel used for wireless communication between a headset and base station.

```
public static uint GetNetworkChannel(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the current network channel for.
deviceName	The name of the device to retrieve the current network channel number for.

Returns

The number of the current network channel.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

The network channel number specifies the channel that is used for communication between a headset and base station. When switching on a g.Nautilus device, it pairs itself automatically to a base station using the first free channel selected by the base station. A free channel is a channel where no communication is running.

GetScaling

Returns the scaling values for the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

```
public static GdsGNautilusScaling GetScaling(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the scaling values for.
deviceName	The name of the device to retrieve the scaling values for.

Returns

The scaling values for the specified g.Nautilus device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

Scaling contains an offset value and scaling factor for each channel of the specified g.Nautilus device.

SetScaling

Sets the scaling values for the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

```
public static void SetScaling(UInt64 connectionHandle, string deviceName, GdsGNautilusScaling scaling)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	The name of the device to set the scaling values for.
scaling	The scaling values to set.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

In neutral scaling, all offset values are set to 0 and all scaling factors are set to 1.

Calibrate

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions if the calibration set is applied.

```
public static GdsGNautilusScaling Calibrate(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to calibrate.
deviceName	The name of the g.Nautilus device that is to be calibrated.

Returns

The new scaling values calculated under the current conditions for each synchronously sampled analog input channel.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running.

Remarks

The returned values will not be applied unless they're explicitly set with [SetScaling](#).

Calibration is required if amplitude and/or offset of the known input signal differs from the measured signal.

The function returns `Single.NaN` as scaling factor and offset for channels that failed calibration.

GetBandpassFilters

Retrieves a list of available bandpass filters for the specified g.Nautilus device.

```
public static GdsFilterInfo[] GetBandpassFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the filter list for.
deviceName	The name of the device to retrieve the filter list for.

Returns

A list of bandpass filters available for the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetNotchFilters

Retrieves a list of available notch filters for the specified g.Nautilus device.

```
public static GdsFilterInfo[] GetNotchFilters(UInt64 connectionHandle, string deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to retrieve the filter list for.
deviceName	The name of the device to retrieve the filter list for.

Returns

A list of notch filters available for the device.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

GetImpedance

Measures and retrieves the impedance of all available synchronously sampled analog input channels of a g.Nautilus device in ohm.

This method is deprecated (use [GetImpedanceEx](#) instead).

```
public static IDictionary<ushort, double> GetImpedance(UInt64 connectionHandle, string
deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.

Returns

A dictionary containing zero-based channel numbers in its keys and the corresponding impedance value in ohm in its values.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running, or the device does not support impedance measurement, or synchronous or asynchronous impedance measurement is currently running, or an invalid reference signal is measured on channel 'Cz' (maybe due to a not connected electrode).

Remarks

Electrode 'Cz' plays a special role in impedance measurement and must always be connected. The method will throw an exception if an invalid reference signal is measured on electrode 'Cz' (maybe due to an unconnected electrode), which does not allow calculation of impedance values of all other channels.

All returned impedance values are rounded to the nearest integral kilo-ohm value. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. However, if the impedance value cannot be measured exactly, the method returns a value of 4,000 ohm for any estimated impedance value below 5,000 ohm, and 501,000 for any estimated impedance value above 500,000 ohm. The returned map only includes available channels that yielded a finite impedance value.

GetImpedanceEx

Measures and retrieves the impedance of all available synchronously sampled analog input channels of a g.Nautilus device in ohm.

```
public static IDictionary<ushort, double> GetImpedanceEx(UInt64 connectionHandle, string
deviceName)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.

Returns

A dictionary containing zero-based channel numbers in its keys and the corresponding impedance value in ohm in its values.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle, or data acquisition is currently running, or the device does not support impedance measurement, or synchronous or asynchronous impedance measurement is currently running.

Remarks

Electrode 'Cz' plays a special role in impedance measurement and must always be connected. The method will not throw an exception if an invalid reference signal is measured on electrode 'Cz' (maybe due to an unconnected electrode). Instead, appropriate error codes will be assigned to the other channels.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The returned map only includes available channels that yielded a finite impedance value.

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too large.
-7	The shape of the measured signal does not match the expected one.
-8	Too many invalid data frames were acquired during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.
-9	The amplitude of the measured signal at electrode 'Cz' is too small.
-10	The offset of the measured signal at electrode 'Cz' is too big.
-11	The shape of the measured signal at electrode 'Cz' does not match the expected one.
-12	Too many invalid data frames were acquired for electrode 'Cz' during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.

StartImpedanceMeasurement

Starts continuous impedance measurement asynchronously and calls the specified callback for each completed measurement cycle.

This method is deprecated (use [StartImpedanceMeasurementEx](#) instead).

```
public static void StartImpedanceMeasurement(UInt64 connectionHandle, string deviceName,
ImpedanceMeasurementCallback impedanceMeasurementCallback, IntPtr userData = default(IntPtr))
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.
impedanceMeasurementCallback	A delegate to the method to execute for each completed asynchronous impedance measurement cycle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

This method returns immediately. The `impedanceMeasurementCallback` function will be called after each completed impedance measurement cycle until [StopImpedanceMeasurement](#) is called.

If an error occurs during impedance measurement, it is forwarded to the callback as parameters. An error occurs whenever [GetImpedance](#) might throw a [GdsException](#).

If the `result` parameter in the [ImpedanceMeasurementCallback](#) callback handler indicates an error and the `aborted` flag of the callback handler equals `true` as well, impedance measurement has been stopped automatically and can be started again with [StartImpedanceMeasurement](#).

If the `result` parameter in the [ImpedanceMeasurementCallback](#) callback handler indicates an error but the `aborted` flag in the callback handler equals `false`, only the current measurement cycle is affected, but impedance measurement continues.



IMPORTANT

`impedanceMeasurementCallback` must be prevented from garbage collection as long as impedance measurement is running (i.e. as long as the callback may be triggered, which is until [StopImpedanceMeasurement](#) is called and has completed for `connectionHandle` and `deviceName`). The recommended way to do this is to store a reference to the delegate passed to `impedanceMeasurementCallback` in a scope that exists until [StopImpedanceMeasurement](#) is called and has completed for `connectionHandle` and `deviceName`.

StartImpedanceMeasurementEx

Starts continuous impedance measurement asynchronously and calls the specified callback for each completed measurement cycle.

```
public static void StartImpedanceMeasurementEx(UInt64 connectionHandle, string deviceName,
ImpedanceMeasurementCallback impedanceMeasurementCallback, IntPtr userData = default(IntPtr))
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.
impedanceMeasurementCallback	A delegate to the method to execute for each completed asynchronous impedance measurement cycle.
userData	A pointer to an unmanaged block of memory containing custom data which is stored and returned when the callback is triggered, or IntPtr.Zero if not used.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

Remarks

This method returns immediately. The `impedanceMeasurementCallback` function will be called after each completed impedance measurement cycle until [StopImpedanceMeasurement](#) is called.

If an error occurs during impedance measurement, it is forwarded to the callback as parameters. An error occurs whenever [GetImpedanceEx](#) might throw a [GdsException](#).

If the `result` parameter in the [ImpedanceMeasurementCallback](#) callback handler indicates an error and the `aborted` flag of the callback handler equals `true` as well, impedance measurement has been stopped automatically and can be started again with [StartImpedanceMeasurementEx](#).

If the `result` parameter in the [ImpedanceMeasurementCallback](#) callback handler indicates an error but the `aborted` flag in the callback handler equals `false`, only the current measurement cycle is affected, but impedance measurement continues.



IMPORTANT

`impedanceMeasurementCallback` must be prevented from garbage collection as long as impedance measurement is running (i.e. as long as the callback may be triggered, which is until [StopImpedanceMeasurement](#) is called and has completed for `connectionHandle` and `deviceName`). The recommended way to do this is to store a reference to the delegate passed to `impedanceMeasurementCallback` in a scope that exists until [StopImpedanceMeasurement](#) is called and has completed for `connectionHandle` and `deviceName`.

StopImpedanceMeasurement

Stops continuous, asynchronous impedance measurement started with [StartImpedanceMeasurement](#) before.

```
public static void StopImpedanceMeasurement(UInt64 connectionHandle, string deviceName, bool waitUntilCompleted)
```

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to measure impedance for.
deviceName	The name of the device to measure impedance for.
waitUntilCompleted	true to block until impedance measurement has been completed; false to return immediately.

Exceptions

ArgumentException	deviceName contains more than GdsClientApiLibraryWrapper.DeviceNameLengthMax characters.
GdsException	connectionHandle is invalid, or connectionHandle is not associated with a data acquisition session opened with Connect before, or deviceName refers to a device that is not part of the data acquisition session associated with connectionHandle.

g.NEEDACCESS
NETWORK ENABLED EASY DATA ACCESS